

La Programmazione ad Oggetti in Python

Docente: Ambra Demontis

Anno Accademico: 2025 - 2026

Corso di Laurea in Ingegneria Elettronica, Informatica e delle Telecomunicazioni



University of Cagliari,
Italy

Department of Electrical and
Electronic Engineering



La Programmazione ad Oggetti in Python

In queste slide vedremo:

- Polimorfismo
- Duck Typing
- Classi Astratte

Polimorfismo

In Python, la stessa classe non può avere più metodi con lo stesso nome.

```
class CStudente:
```

```
    def __init__(self, nome, cognome):
```

```
        self.nome = nome
```

```
        self.cognome = cognome
```

Non si può fare!

```
    def __init__(self):
```

```
        self.nome = input("inserisci il nome ")
```

```
        self.cognome = input("inserisci il cognome ")
```

Polimorfismo

Classi diverse però possono avere funzioni con lo stesso nome...

```
class CAzienda(CCliente):  
    def stampa_telefono(self):  
        print("Il numero di telefono dell'azienda con ", self.idx, " è: ",  
              self.numero_telefono)  
  
class CPersona(CCliente):  
    def stampa_telefono(self):  
        print("Il numero di telefono della persona con ", self.idx, " è: ",  
              self.numero_telefono)
```

Polimorfismo

```
class CCliente:
```

```
    def __init__(self, idx, numero_telefono):
```

```
        self.idx = idx
```

```
        self.numero_telefono = numero_telefono
```

```
oggetto_azienza = CAzienda("123", "070998899")
```

```
oggetto_persona = CPersona("422", "338786544")
```

Polimorfismo

Il codice che utilizza gli oggetti appartenenti a queste classi, per stampare il numero di telefono non ha bisogno di sapere se l'oggetto appartiene alla classe azienda o alla classe persona.

Dovrà semplicemente richiamare la funzione stampa_telefono dell'oggetto e il comportamento di questa funzione sarà differente a seconda della classe di appartenenza dell'oggetto.

Polimorfismo

`oggetto_azienda.stampa_telefono()`

`oggetto_persona.stampa_telefono()`

Stamperà:

Il numero di telefono dell'azienda con 123 è: 070998899

Il numero di telefono della persona con 123 è: 070998899

Polimorfismo

Il fatto che lo stesso codice di comportamenti in modo differente **a seconda della sottoclasse** di appartenenza viene detto **polimorfismo**.

Duck Typing

In Python, per poter scrivere un unico codice che può trattare diverse classi e che funziona in modo differente a seconda della classe di appartenenza dell'oggetto, **non è necessario che queste classi siano figlie della stessa classe.**

Basta che queste classi **abbiano la stessa interfaccia!**

Questa proprietà viene detta **duck typing**.

“Se cammina come un’anatra e nuota come un’anatra, allora è un’anatra.”
(Se la classe ha l’interfaccia che mi serve, allora posso utilizzarla).

Duck Typing

Supponete di voler creare un programma di contabilità per un piccolo negozio. Il gestore, nel periodo degli sconti, applica uno sconto differente in base al prodotto ma sempre uguale per lo stesso prodotto.

Duck Typing

Realizzate quindi il seguente codice:

```
class CQuadernoniPigna():  
    prezzo = 3  
    @classmethod  
    def calcola_prezzo_scontato(cls):# 10 % sconto  
        return cls.prezzo * 10 / 100
```

```
class CCaffePaulistaOro():  
    prezzo = 5  
    @classmethod  
    def calcola_prezzo_scontato(cls): # sottrai sempre 2 euro al prezzo  
        return cls.prezzo - 2
```

Il codice che utilizza queste classi potrà sfruttare il duck typing: diverse classi (non parenti) che espongono un metodo con la stessa interfaccia.

Le Classi Astratte

C'è però un problema!

Supponete che il codice per la gestione di questo programma diventi molto grande e complesso (con tante classi, tanti metodi ecc.)

Supponete anche che venga assunto un altro programmatore che dovrà darvi una mano creando nuove classi.

Come fa a sapere quali interfacce deve rispettare?

Le Classi Astratte

Una soluzione potrebbe essere fornirgli i diagrammi delle classi di tutto il codice...

Nella pratica questo spesso non è fattibile.

I diagrammi di classe generalmente vengono utilizzati per definire i prototipi.

E' raro che vengano mantenuti aggiornati quando un progetto molto grande viene modificato!

Le Classi Astratte

Esiste però un meccanismo ad-hoc: le **classi astratte**.

Servono a definire l'interfaccia che altre classi dovranno avere.

Le Classi Astratte

Supponiamo di volere creare una classe astratta per il codice:

```
class CQuadernoniPigna(CProdotto):
```

```
    prezzo = 3
```

```
    @classmethod
```

```
    def calcola_prezzo_scontato(cls):# 10 % sconto
```

```
        return cls.prezzo * 10 / 100
```

```
class CCaffePaulistaOro(CProdotto):
```

```
    prezzo = 5
```

```
    @classmethod
```

```
    def calcola_prezzo_scontato(cls): # sottrai sempre 2 euro al prezzo
```

```
        return cls.prezzo - 2
```

Le Classi Astratte

```
from abc import ABC, abstractmethod
```

```
class CProdotto(ABC):
```

```
    @classmethod
```

```
    @abstractmethod
```

```
    def calcola_prezzo_scontato(cls):  
        pass
```

Il nuovo programmatore, quando guarda questa classe astratta sa che la nuova classe che deve implementare dovrà avere un metodo chiamato *calcola_prezzo_scontato* che non riceve nessun argomento.

Le Classi Astratte

Se dovesse comunque creare una classe che non ha quel metodo, quando un oggetto di quella classe verrà istanziato verrà sollevata un'eccezione.

Es:

```
class CCacaoCraiAmaro(CProdotto):  
    pass
```

```
oggetto_cacao = CCacaoCraiAmaro()
```

**TypeError: Can't instantiate abstract class CCacao with abstract methods
calcola_prezzo_scontato**

Le Classi Astratte

```
from abc import ABC, abstractmethod
```

```
class CProdotto(ABC):  
    def __init__(self, codice):  
        self.codice_prodotto = codice
```

```
@classmethod
```

```
@abstractmethod
```

```
def calcola_prezzo_scontato(cls):  
    pass
```

Le classi astratte possono anche implementare dei metodi.

Esercizio sull'uso delle Classi Astratte

Volete creare delle classi per gestire delle forme geometriche.

In questo esercizio volete gestire il rettangolo e il cerchio ma sapete che in futuro dovranno essere implementate altre forme geometriche.

Per il rettangolo dovrete acquisire in input larghezza e altezza,
Per il cerchio il raggio.

Per ogni forma geometrica il programma deve permettere di calcolare l'area e il perimetro.

Per il π potete utilizzare la costante pi definita dal modulo math.

Esercizio sull'uso delle Classi Astratte

```
from math import pi

class CCerchio(CFormaGeometrica):

    def __init__(self, raggio):
        self.raggio = raggio

    @classmethod
    def crea_figura_input_tastiera(cls):
        raggio = input("inserisci il raggio della figura ")
        return cls(raggio)

    def calcola_perimetro(self):
        return 2 * self.raggio * pi

    def calcola_area(self):
        return pi * self.raggio ** 2
```

Esercizio sull'uso delle Classi Astratte

```
class CRettangolo(CFormaGeometrica):  
  
    def __init__(self, larghezza, altezza):  
        self.larghezza = larghezza  
        self.altezza = altezza  
  
    @classmethod  
    def crea_figura_input_tastiera(cls):  
        larghezza = input("inserisci la larghezza ")  
        altezza = input("inserisci l'altezza")  
        return cls(larghezza, altezza)  
  
    def calcola_perimetro(self):  
        return (self.larghezza + self.altezza) * 2  
  
    def calcola_area(self):  
        return self.larghezza * self.altezza
```

Esercizio sull'uso delle Classi Astratte

```
from abc import ABC, abstractmethod

class CFormaGeometrica(ABC):

    @classmethod
    @abstractmethod
    def crea_figura_input_tastiera(cls):
        pass

    @abstractmethod
    def calcola_perimetro(self):
        pass

    @abstractmethod
    def calcola_area(self):
        pass
```